

1. In the structure of the producer process shown below, what would be a possible outcome if `wait(empty)` is replaced with `signal(empty)` and `signal(full)` is replaced with `wait(full)`?

Note that the producer and consumer processes share the following data structures: `int n;`

```
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0

while (true) {
    . . .
    /* produce an item in next_produced */
    . . .
    wait(empty);
    wait(mutex);

    . . .
    /* add next_produced to the buffer */
    . . .
    signal(mutex);
    signal(full);
}
```

- A) Producer will remain blocked after adding an item in the buffer.
B) The buffer will always remain empty.
C) Producer and consumer may access the buffer at the same time.
D) All of the above.
2. The classic readers-writers problem involves coordinating access to a shared data set such that multiple readers can access it simultaneously, but writers require exclusive access. In the reader-priority solution, readers can enter freely as long as no writer is currently writing. This is typically implemented using a `read_count` variable and semaphores `mutex` and `rw_mutex`. In this setup:
- `mutex` protects access to `read_count`.
 - `rw_mutex` ensures exclusive access for writers.
 - The first reader locks out writers, and the last reader lets them in.
- a. Explain how this solution can lead to starvation of writers.
b. Suggest a modification or mechanism to reduce or eliminate this starvation, and briefly explain how it works. You may use brief pseudocode if it helps support your explanation.
3. Is it possible to have a deadlock involving only one single-threaded process? Explain your answer.

4. In the classical Dining-Philosophers problem, each philosopher alternates between thinking and eating. The basic solution using semaphores involves each philosopher executing:

```
1 wait(chopstick[i]);
2 wait(chopstick[(i+1)%5]);
3
4 /* eat */
5
6 signal(chopstick[i]);
7 signal(chopstick[(i+1)%5]);
```

- a. Explain how deadlock can occur in this solution.
- b. Suggest a strategy to avoid deadlock. Clearly describe your approach in structured steps or a few key code-like lines; complete pseudocode is not required.
5. Suppose a system with two types of processes: type A and type B. All processes of type A execute the same code, and all processes of type B execute the same code. The code for each process type is shown in the table below.

A Processes	B Processes
wait(X) signal(Y)	wait(Y) wait(Y) signal(X) signal(Y)

Assume X and Y are general semaphores, where X is initialized to 2 and Y is initialized to 0. Suppose three processes of type A and two of type B are brought into execution simultaneously. Answer the two questions below:

- A. Can the processes finish in the order AABAB? If yes, provide an execution sequence that achieves this order. If no, explain why in detail.
- B. Can processes finish in the order AABBA? If yes, provide an execution sequence that achieves this order. If no, provide a detailed explanation.
6. Given a 22-bit logical address and a 64 KB page size, how is the logical address divided into the page number and offset in a single-level page table?

7. Consider a 24-bit address space and an 8 KB page size. The single-level page table uses only half a page (4 KB). Based on this information:
- How many pages are in the address space?
 - What is the size of each page table entry (PTE)?
 - Hint: Calculate the number of pages in the address space first, then use the total size of the page table and the number of entries to determine the size of each PTE.
8. Suppose we have the following resources: R1, R2, and R3 and threads T1, T2, and T3. The **maximum** number of each resource available is:

Total Resources Available		
R1	R2	R3
8	5	3

Assume that the processes have the following maximum requirements and current allocations:

Thread ID	Current Allocation				Maximum Requirements		
	R1	R2	R3		R1	R2	R3
T1	2	1	0		4	1	1
T2	2	0	1		3	4	3
T3	1	1	1		5	2	1

The above state is safe. Consider each of the following requests and say if they can be granted. **Assume each request starts from the above safe state.** Show your work by applying the Banker's algorithm.

1. T3 makes a request for additional resources: $(R1, R2, R3) = (2, 0, 0)$. Can the request be granted?
2. T2 makes a request for additional resources: $(R1, R2, R3) = (0, 2, 0)$. Can the request be granted?
3. T1 makes a request for additional resources: $(R1, R2, R3) = (0, 0, 1)$. Can the request be granted?

Good luck!